



UNIVERSIDAD DE LA CUENCA DEL PLATA
Facultad de Ingeniería y Tecnología — Sede Corrientes
Ingeniería en Sistemas de Información

Simulador de procesos y memoria

Informe técnico — Actividad de Evaluación 2 (intercátedra)
Paradigmas y Lenguajes de Programación II · Sistemas Operativos

Alumno	Fabricio Barreto
DNI	43.023.002
Docentes de Sistemas Operativos	Mirta Luque · Patricia Daniela Montiel
Docente de Paradigmas II	José A. Fernández
Repositorio	github.com/FabricioBarreto/simulador-procesos-memoria
Versión evaluada	tag <code>v1.0</code>
Fecha	5 de octubre de 2026

Índice

1. Introducción
2. Objetivos
3. Escenario y alcance
4. Descripción general de la solución
5. Diseño: clases y responsabilidades
6. Decisiones de programación orientada a objetos
7. Estructuras de datos utilizadas
8. Estados de los procesos: diagrama de transición y diagrama de flujo
9. Orden de cada tick y planificación Round Robin
10. Gestión de memoria: política de asignación y coalescencia
11. Diagramas de secuencia
12. Matriz de trazabilidad RF → clase → tests
13. Casos de prueba y resultados
14. Cobertura de código
15. Análisis comparativo First-Fit, Best-Fit y Worst-Fit
16. Análisis de la fragmentación externa
17. Conclusiones
18. Bibliografía y fuentes
19. Anexo: cómo reproducir los resultados

1. Introducción

Un sistema operativo tiene que repartir dos recursos escasos entre muchos procesos: la **memoria principal** y la **CPU**. Este trabajo modela ese reparto con una biblioteca de clases escrita en **TypeScript** que simula, en unidades discretas de tiempo (*ticks*), cómo los procesos piden memoria contigua, esperan su turno en una cola, usan la CPU por turnos con **Round Robin**, se bloquean por operaciones de Entrada/Salida y liberan sus recursos al terminar.

La biblioteca no tiene interfaz gráfica, menú ni función `main`: todo su comportamiento se demuestra con **217 pruebas automatizadas** (Vitest) que pasan en su totalidad y cubren el **100%** del código de producción. El informe documenta el diseño orientado a objetos, las decisiones tomadas, los resultados de las pruebas y un análisis comparativo de las tres políticas de asignación de memoria.

2. Objetivos

Objetivo general

Transformar los requerimientos de un simulador de Sistemas Operativos en un modelo de clases cohesivo, encapsulado y verificable, aplicando programación orientada a objetos.

Objetivos específicos

- Representar el ciclo de vida de un proceso con seis estados y transiciones controladas.
- Implementar la asignación contigua de memoria con las políticas **First-Fit**, **Best-Fit** y **Worst-Fit**, intercambiables por polimorfismo.
- Implementar la liberación de memoria con **coalescencia** automática de bloques libres adyacentes.
- Planificar la CPU con **Round Robin** y quantum configurable, contando los cambios de contexto según la convención de la consigna.
- Simular bloqueos por E/S deterministas.
- Calcular métricas de ocupación, utilización de CPU y **fragmentación externa**.
- Verificar cada requerimiento con pruebas automatizadas y cobertura mayor al 90%.
- Analizar e interpretar el comportamiento de las políticas de asignación y la fragmentación que producen.

3. Escenario y alcance

Un laboratorio universitario necesita un motor de simulación para estudiar cómo los procesos comparten una memoria limitada y una única CPU. La simulación comienza en el tick 0, con memoria vacía y CPU libre. La configuración de referencia es **1024 KB de memoria** y **quantum de 2 ticks**; ambos parámetros son configurables, igual que la política de asignación.

Alcance: la solución es una biblioteca de clases. No incluye interfaz gráfica ni salida por consola: los métodos devuelven resultados y estados consultables, y la evidencia de funcionamiento son los tests. El avance es determinista: no hay esperas reales, temporizadores, hilos ni aleatoriedad.

4. Descripción general de la solución

Aspecto	Decisión
Lenguaje	TypeScript en modo estricto (<code>strict</code> y <code>noImplicitOverride</code> , que obliga a marcar con <code>override</code> cada método redefinido).
Pruebas y cobertura	Vitest 5 con el proveedor <code>@vitest/coverage-v8</code> . Umbral configurado: el comando falla si la cobertura de líneas baja del 90%.
Organización	<code>src/</code> (código de producción) y <code>tests/</code> (pruebas) separados. Dentro de <code>src/</code> : <code>interfaces/</code> , <code>modelos/</code> , <code>modelos/estrategias/</code> , <code>errores/</code> y <code>utilidades/</code> .
Versionado	Git y GitHub, con commits pequeños por paso de desarrollo e integración continua con GitHub Actions: cada <code>push</code> corre <code>typecheck</code> y <code>coverage</code> .
Diagramas	draw.io (archivos editables <code>.drawio</code> y versiones PNG en <code>uml/</code>).

La idea central del diseño es que **el simulador coordina pero no hace el trabajo**: la memoria la administra el `GestorMemoria`, la CPU el `PlanificadorRoundRobin` y cada `Proceso` cuida su propio estado. El simulador solo decide el **orden de las fases** de cada tick.

5. Diseño: clases y responsabilidades

Clase / interfaz	Tipo	Responsabilidad
Simulador	Clase (coordinadora)	Configura la simulación, registra procesos, ejecuta las cuatro fases de cada tick y expone el estado del sistema con vistas de solo lectura. Implementa <code>ISimulador</code> .
Proceso	Clase	Mantiene sus datos (PID, memoria, CPU total y restante, quantum consumido, bloqueo) y controla sus propias transiciones de estado.
EstadoProceso	Enumeración	Los seis estados del ciclo de vida.
EventoES	Objeto de valor inmutable	Evento de E/S determinista: después de cuántos ticks de CPU se dispara y cuánto dura.
GestorMemoria	Clase	Mantiene el mapa de bloques ordenado y continuo; asigna con la estrategia recibida, libera y hace coalescencia. Implementa <code>IGestorMemoria</code> .
BloqueMemoria	Objeto de valor inmutable	Tramo contiguo con inicio, tamaño y proceso asignado o libre. Sabe dividirse (<code>asignar</code>), liberarse y fusionarse con su vecino.
IEstrategiaAsignacion	Interfaz	Contrato de las políticas: <code>elegirBloque(bloques, tamaño)</code> .
EstrategiaAsignacionBase	Clase abstracta	Algoritmo común (filtrar libres suficientes, ordenar por dirección, desempatar por menor dirección). Deja abstracto el criterio <code>esMejor()</code> .
FirstFit, BestFit, WorstFit	Subclases	Cada una redefine (<code>override</code>) solo el criterio de "mejor bloque".
PlanificadorRoundRobin	Clase	Cola FIFO de Listos, despacho, quantum y contador de cambios de contexto. Implementa <code>IPlanificador</code> .
Metricas	Objeto de valor inmutable	"Foto" de las seis métricas al final de cada tick.
VistaProceso, ConfiguracionSimulacion	Interfaces	Vista congelada de un proceso para consultas y parámetros opcionales de configuración.
ErrorDominio y 7 subclases	Clase abstracta + subclases	Errores de las reglas del dominio: dato inválido, transición inválida, operación inválida, evento de E/S inválido, operación de memoria inválida, planificación inválida y registro inválido.

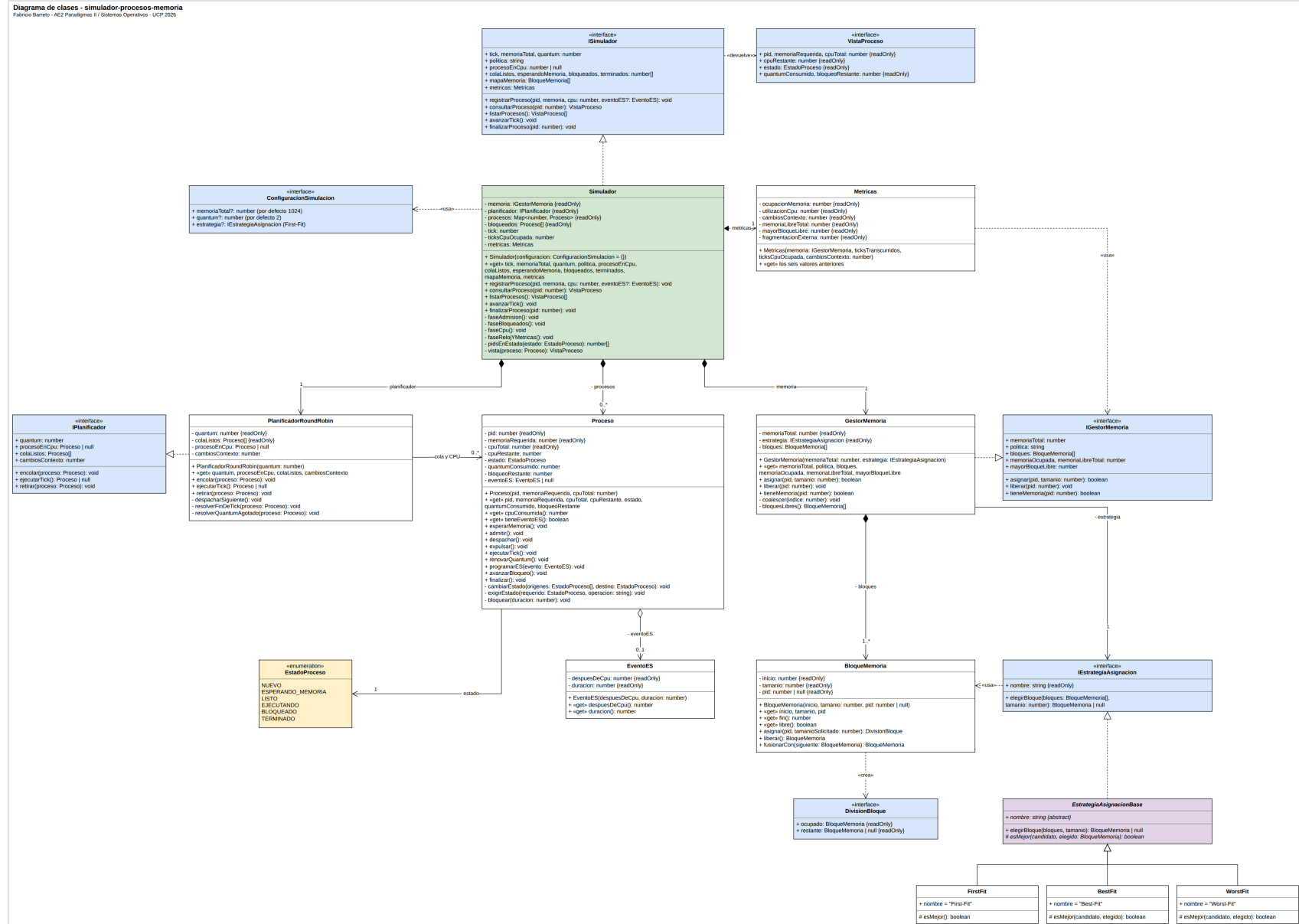


Figura 1. Diagrama de clases: simulador, procesos, planificación y memoria (archivo editable uml/diagrama-clases.drawio).

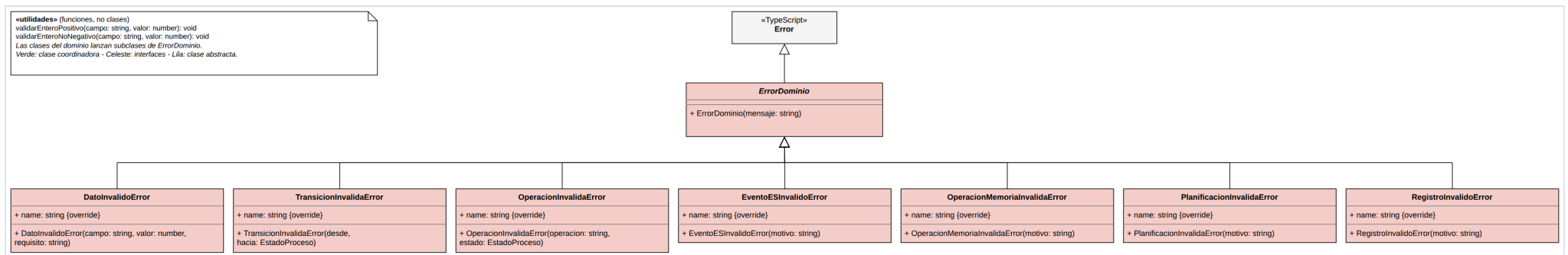


Figura 2. Diagrama de clases (continuación): jerarquía de errores del dominio.

Lectura del diagrama. Rombo relleno = composición (el `simulador` crea sus partes y estas no existen sin él). Rombo vacío = agregación (el `EventoES` se crea afuera y se le pasa al proceso). Flecha punteada con triángulo = realización de una interfaz. Flecha continua con triángulo = herencia. Flecha punteada abierta = dependencia («usa», «crea», «devuelve»). Visibilidad: - privado (atributos # de TypeScript), + público, # protegido. En cursiva, los elementos abstractos.

6. Decisiones de programación orientada a objetos

6.1 Responsabilidades y alta cohesión

Cada clase tiene una sola razón para cambiar (**S** de SOLID). El `simulador` no contiene lógica de memoria ni de CPU: `avanzarTick()` son cuatro llamadas a sus fases privadas, y cada fase delega en la parte que corresponde. Así se evita una clase que concentre toda la solución.

6.2 Doble encapsulamiento

- **Primera capa:** todos los atributos son privados con `#`, que en JavaScript es privacidad real en tiempo de ejecución, no solo del compilador. No hay setters.
- **Segunda capa:** el estado solo cambia con operaciones del dominio que validan las reglas. Por ejemplo, `Proceso.despachar()` solo es válido desde `Listo`; cualquier intento indebido lanza `TransicionInvalidaError`, y todas las transiciones pasan por un único método privado, `#cambiarEstado`.
- **Colecciones protegidas:** los métodos que devuelven listas (`bloques`, `colaListos`, `PIDs`) entregan **copias**. `BloqueMemoria`, `EventoES` y `Metricas` son **inmutables**.
- **Vistas de solo lectura:** el simulador nunca entrega un `Proceso` real, sino una `VistaProceso` congelada con `Object.freeze`. Si entregara el objeto, desde afuera se podría llamar a `despachar()` y saltar al planificador.

Los tests lo verifican: `Reflect.set(proceso, 'estado', TERMINADO)` devuelve `false` y el estado no cambia.

6.3 Herencia y justificación

Se usa herencia solo donde hay una relación "es un" que respeta la sustitución (**L** de SOLID):

- `FirstFit`, `BestFit` y `WorstFit` *son una* `EstrategiaAsignacionBase`: cualquiera puede usarse donde se espera una estrategia.
- Los siete errores *son un* `ErrorDominio`, que *es un* `Error`: se pueden capturar como la clase base sin conocer la subclase, y cada uno redefine `name` con `override`.

Dónde NO se usó: `Proceso` no hereda de nada, porque no existen tipos de proceso que sean casos especiales de otro. Tampoco el `Simulador`, que *tiene* sus partes (composición) en lugar de *ser una* de ellas.

6.4 Clases abstractas

`EstrategiaAsignacionBase` reúne lo que las tres políticas hacen igual (validar el pedido, filtrar bloques libres suficientes, ordenarlos por dirección y recorrerlos) y deja abstracto solo `esMejor(candidato, elegido)`. Es el patrón *Template Method*: el desempate por menor dirección sale solo, porque solo se reemplaza al elegido si el candidato es *estrictamente* mejor. `ErrorDominio` es abstracta porque no tiene sentido lanzar un error del dominio sin especificar cuál.

6.5 Interfaces y principios SOLID

Principio	Dónde se aplica
-----------	-----------------

S - Responsabilidad única	Memoria, CPU, estados, métricas y coordinación están en clases distintas.
O - Abierto/cerrado	Para agregar una política nueva (por ejemplo Next-Fit) basta una subclase con su <code>esMejor()</code> , sin tocar el gestor.
L - Sustitución de Liskov	Las tres estrategias se usan indistintamente a través de <code>IEstrategiaAsignacion</code> , y los tests del contrato común corren igual sobre las tres.
I - Segregación de interfaces	Interfaces chicas y específicas: <code>IEstrategiaAsignacion</code> tiene un solo método; <code>IPlanificador</code> e <code>IGestorMemoria</code> exponen solo lo que el simulador usa.
D - Inversión de dependencias	El <code>Simulador</code> guarda sus partes como <code>IGestorMemoria</code> e <code>IPlanificador</code> ; el <code>GestorMemoria</code> depende de <code>IEstrategiaAsignacion</code> ; <code>Metricas</code> recibe un <code>IGestorMemoria</code> .

6.6 Polimorfismo

El gestor llama a `this.#estrategia.elegirBloque(...)` sin saber qué política es: no existe ningún `if (politica === 'best')`. La política se elige al construir el simulador: `new Simulador({ estrategia: new BestFit() })`. **En los tests**, `describe.each` corre los mismos siete casos del contrato sobre las tres estrategias usándolas como `IEstrategiaAsignacion`, y un test parametrizado muestra que el mismo pedido de 150 KB cae en las direcciones 0, 850 o 300 según la política.

6.7 Composición y colaboración

El `Simulador` crea en su constructor el `GestorMemoria`, el `PlanificadorRoundRobin` y sus `Metricas`, y mantiene sus `Proceso`: son relaciones "tiene un" de composición (rombo relleno). `Proceso` tiene un `EventoES` opcional (agregación 0..1). El `GestorMemoria` se compone de sus bloques (1..*) y tiene una estrategia (1).

6.8 Colecciones de objetos

Colección	Dónde	Multiplicidad en UML	Cómo se protege
<code>Map<number, Proceso></code>	Simulador	0..*	Privado; se consulta con vistas congeladas
<code>Proceso[]</code> (bloqueados)	Simulador	0..*	Privado; se expone como lista nueva de PIDs
<code>Proceso[]</code> (cola de Listos)	PlanificadorRoundRobin	0..*	Privado; <code>colaListos</code> devuelve una copia
<code>BloqueMemoria[]</code>	GestorMemoria	1..*	Privado; <code>bloques</code> devuelve una copia de bloques inmutables

7. Estructuras de datos utilizadas

- **Mapa de memoria:** lista de `BloqueMemoria` ordenada por dirección. Invariantes: los bloques son contiguos (cada uno empieza donde termina el anterior), no se solapan, suman la memoria total y nunca hay dos libres juntos. Asignar reemplaza un bloque por una o dos partes (ocupado y restante); liberar lo reemplaza por un bloque libre y lo fusiona con sus vecinos.
- **Cola de Listos:** arreglo usado como cola FIFO: `push` agrega al final y `shift` despacha desde el principio.
- **Procesos registrados:** un `Map` por PID, que permite buscar en tiempo constante, detectar PID repetidos y además conserva el **orden de registro**, que es el orden de admisión que pide RF03.
- **Lista de bloqueados:** arreglo en orden de bloqueo; en la fase 2 los que terminan su E/S vuelven al final de la cola de Listos en ese orden.
- **Objetos de valor:** `EventoES`, `BloqueMemoria`, `Metricas` y la `DivisionBloque` que devuelve `asignar`.

8. Estados de los procesos: diagrama de transición y diagrama de flujo

Estado	Significado
NUEVO	Recién registrado; todavía no intentó conseguir memoria.
ESPERANDO_MEMORIA	No había un bloque contiguo suficiente; se reintentará al inicio de cada tick.
LISTO	Tiene memoria asignada y espera en la cola FIFO para usar la CPU.
EJECUTANDO	Está usando la CPU (como máximo uno por tick).
BLOQUEADO	Espera que termine su E/S; libera la CPU pero conserva la memoria.
TERMINADO	Consumió toda su CPU (o se lo finalizó a la fuerza); su memoria se liberó y no vuelve a ninguna cola.

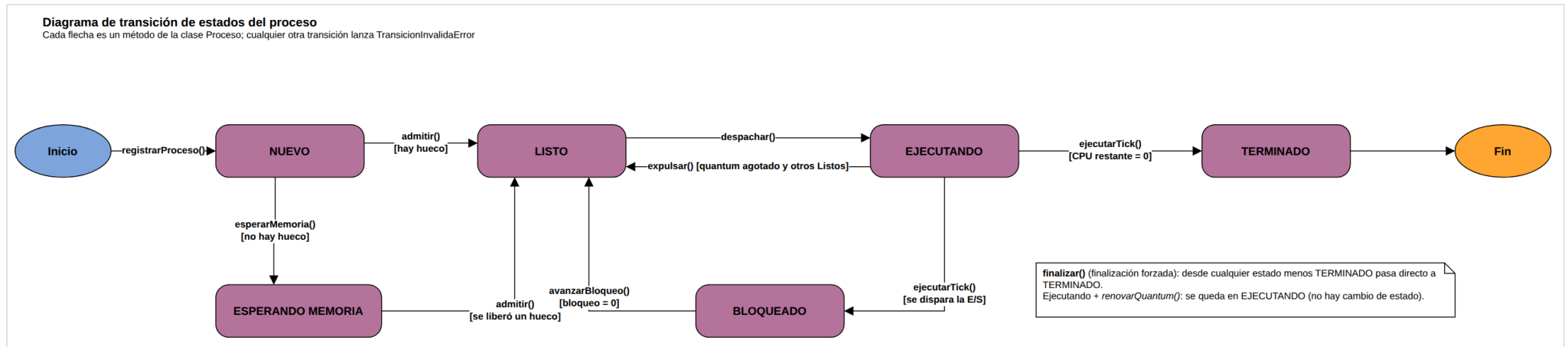


Figura 3. Diagrama de transición de estados: cada flecha es un método de `Proceso` con su condición entre corchetes; cualquier otra transición lanza `TransicionInvalidaError` (archivo editable `uml/diagrama-estados.drawio`).

La finalización forzada (`finalizar()`) lleva a `TERMINADO` desde cualquier estado y se usa para provocar la liberación de un proceso en la defensa. Como complemento, en la página siguiente el diagrama de flujo muestra el mismo ciclo de vida como una secuencia de pasos y decisiones, tal como los evalúa el simulador en cada tick.

Las decisiones del diagrama de flujo se evalúan en el mismo orden de prioridad que el código: primero si terminó su CPU, después si se dispara la E/S y por último si agotó el quantum.

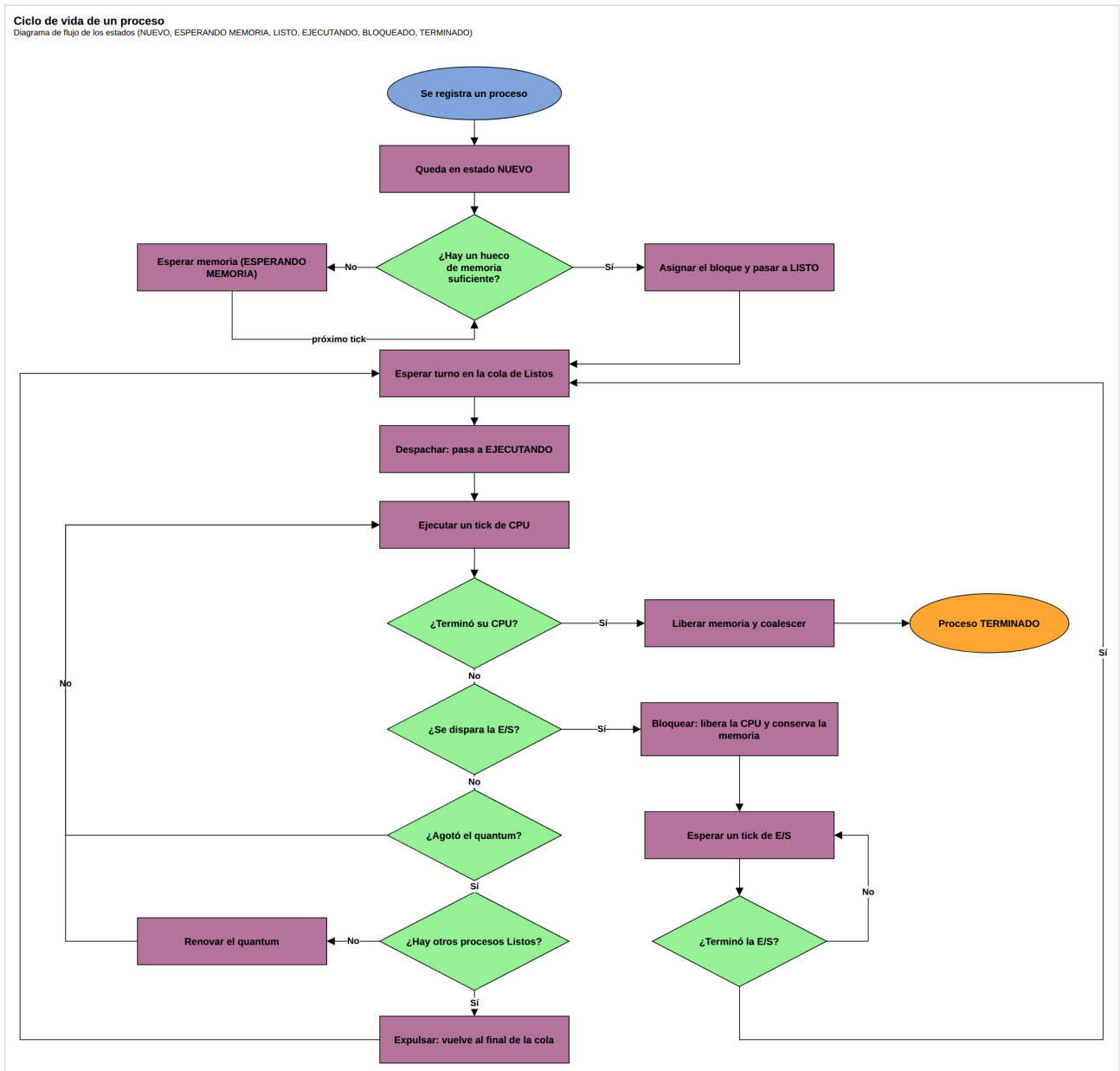


Figura 4. Diagrama de flujo del ciclo de vida de un proceso: cada caja es una operación de Proceso y cada rombo una decisión del simulador (archivo editable `uml/diagrama-flujo-proceso.drawio`).

9. Orden de cada tick y planificación Round Robin

`Simulador.avanzarTick()` avanza **exactamente una** unidad de tiempo, siempre en el mismo orden:

1. **Admisión:** los procesos Nuevos y Esperando Memoria intentan conseguir memoria, en orden de registro. Si uno no entra, se sigue con el siguiente (no frena a los que sí caben).
2. **Bloqueados:** baja el temporizador de E/S de cada bloqueado. Los que llegan a cero vuelven al **final** de la cola de Listos y pueden ejecutarse en ese mismo tick.
3. **CPU (Round Robin):** si la CPU está libre se despacha el primero de la cola (con el quantum en cero) y el proceso consume una unidad de CPU. Después se resuelve con esta prioridad:
 - **terminó** (CPU restante = 0): libera la CPU y su memoria en ese tick; otro proceso entra recién en el tick siguiente;
 - **se bloqueó** por E/S: libera la CPU, conserva la memoria y cuenta un cambio de contexto;
 - **agotó el quantum:** si hay otros Listos, vuelve al final de la cola y cuenta un cambio de contexto; si no hay otros, renueva el quantum y sigue sin cambio de contexto.
4. **Reloj y métricas:** el tick avanza y se calcula una nueva foto de las métricas.

Convención de cambios de contexto (RF09): se cuenta uno por expulsión por quantum con otros Listos y uno por bloqueo de E/S. No se cuentan el despacho inicial, la finalización ni la renovación del quantum sin otros Listos.

Ejemplo (caso mínimo "Round Robin"): con $Q = 2$, P1 con CPU 3 y P2 con CPU 2 la traza es P1, P1, P2, P2, P1. En el tick 2, P1 agota el quantum con P2 esperando y se lo expulsa (único cambio de contexto). En el tick 4, P2 termina justo con el quantum: la finalización tiene prioridad y no se cuenta cambio.

10. Gestión de memoria: política de asignación y coalescencia

10.1 Asignación contigua

Se implementaron las **tres** políticas y se elige una al configurar el simulador (por defecto First-Fit). Todas consideran solo bloques libres de tamaño suficiente y, ante empate, eligen la menor dirección:

- **First-Fit**: el primer bloque suficiente recorriendo por dirección.
- **Best-Fit**: el bloque suficiente de **menor** tamaño (deja el sobrante más chico).
- **Worst-Fit**: el bloque suficiente de **mayor** tamaño (deja el sobrante más grande).

Si sobra espacio, el bloque se divide en una parte ocupada al principio y un bloque libre con el resto; un ajuste exacto no genera bloques de tamaño cero. Si ningún hueco alcanza, la asignación devuelve `false` y **no modifica** los bloques, aunque la suma de la memoria libre alcance (fragmentación externa).

10.2 Liberación y pseudocódigo de la coalescencia

```
procedimiento liberar(pid):
    i ← posición del bloque cuyo pid es el indicado
    si no existe: error "el proceso no tiene memoria asignada"
    bloques[i] ← bloque libre con el mismo inicio y tamaño
    coalescer(i)

procedimiento coalescer(i):
    // 1) vecino derecho
    si existe bloques[i+1] y está libre:
        reemplazar bloques[i] y bloques[i+1] por un único bloque libre
        con inicio = bloques[i].inicio y tamaño = suma de ambos
    // 2) vecino izquierdo (el índice i sigue apuntando al bloque recién liberado)
    si existe bloques[i-1] y está libre:
        reemplazar bloques[i-1] y bloques[i] por un único bloque libre
        con inicio = bloques[i-1].inicio y tamaño = suma de ambos
```

Como la coalescencia se aplica en **cada** liberación, nunca quedan dos huecos libres juntos, así que alcanza con mirar un vecino de cada lado. La coalescencia **no mueve bloques ocupados**: no es compactación. Al liberar todos los procesos queda un único bloque libre del tamaño total.

Ejemplo paso a paso (Caso 3 de las pruebas de colaboración)

Memoria de 1000 KB con cuatro procesos de 250 KB y quantum 1. Con CPU 3, 1, 2 y 4 los procesos terminan en el orden P2, P3, P1, P4, y cada liberación cubre un caso distinto de la coalescencia. Los mapas son los que verifica el test *"Fusión con el vecino izquierdo, con el derecho y al final un único bloque"*.

Inicio: los cuatro procesos admitidos, sin huecos.

P1 250	P2 250	P3 250	P4 250
-----------	-----------	-----------	-----------

Tick 2, termina P2: sus dos vecinos están ocupados, así que no hay fusión y queda un hueco aislado de 250 KB.

P1 250	libre 250	P3 250	P4 250
-----------	--------------	-----------	-----------

Tick 6, termina P3: se fusiona con el hueco de su **izquierda** y forma un bloque libre de 500 KB que empieza en 250.



Tick 8, termina P1: se fusiona con el hueco de su **derecha** y forma un bloque libre de 750 KB que empieza en 0.



Termina P4: se fusiona a la izquierda y la memoria vuelve a ser un único bloque libre de 1000 KB.



El caso de **ambos vecinos** lo cubre el segundo test del Caso 3: con CPU 1, 3, 1 y 5 terminan primero P1 y P3, y al terminar P2 sus huecos de los dos lados se fusionan en un solo paso.

Antes de liberar P2: huecos a ambos lados.



Después de liberar P2: primero se une con el vecino derecho y después con el izquierdo, y queda un único bloque libre de 750 KB.



11. Diagramas de secuencia

Se presentan tres diagramas, asociados a requerimientos distintos, con fragmentos `loop`, `alt` y `opt` coherentes con el código (archivo editable `uml/diagramas-secuencia.drawio`, una página por diagrama).

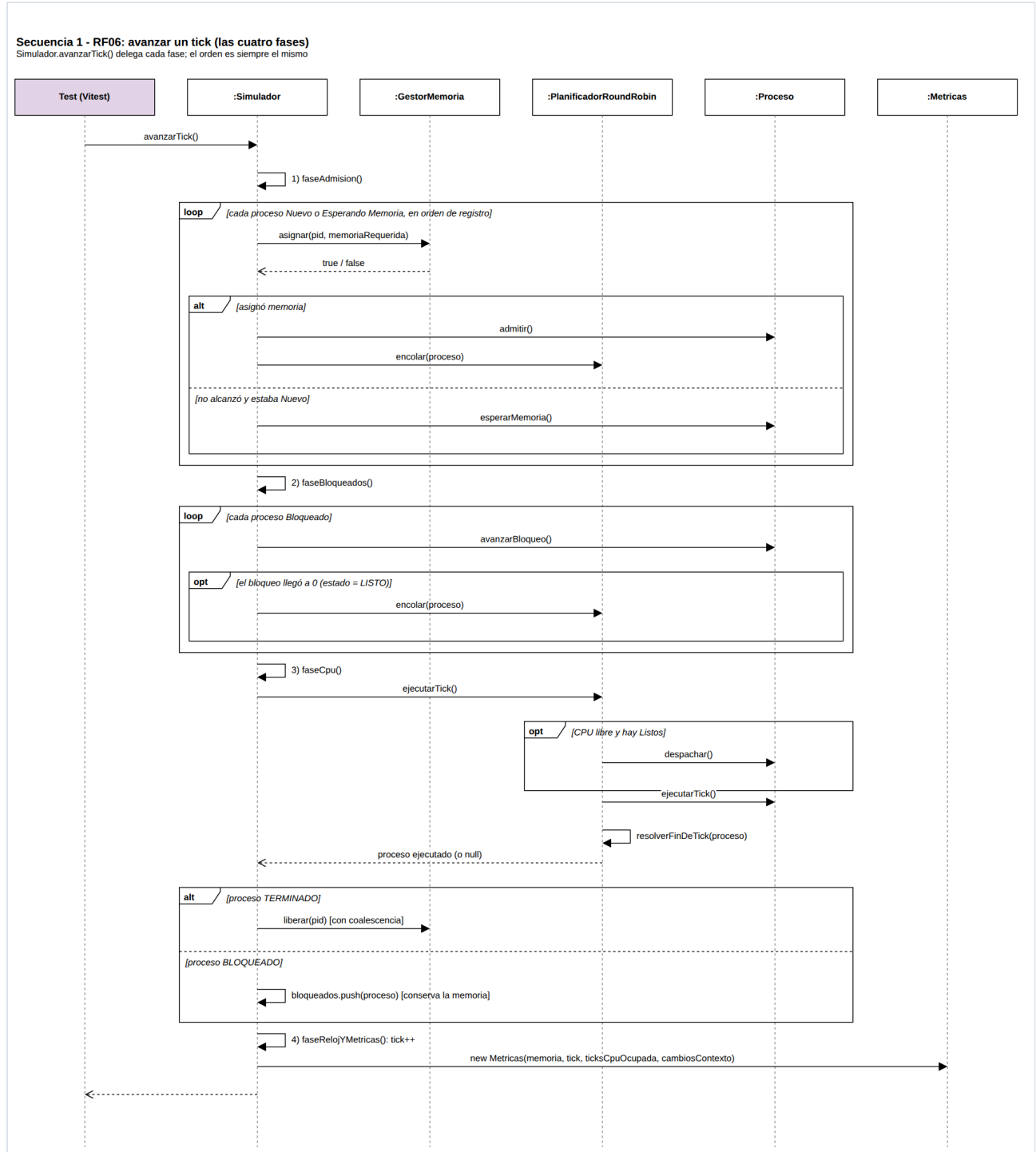


Figura 5. Secuencia 1 — RF06: avanzar un tick (las cuatro fases).

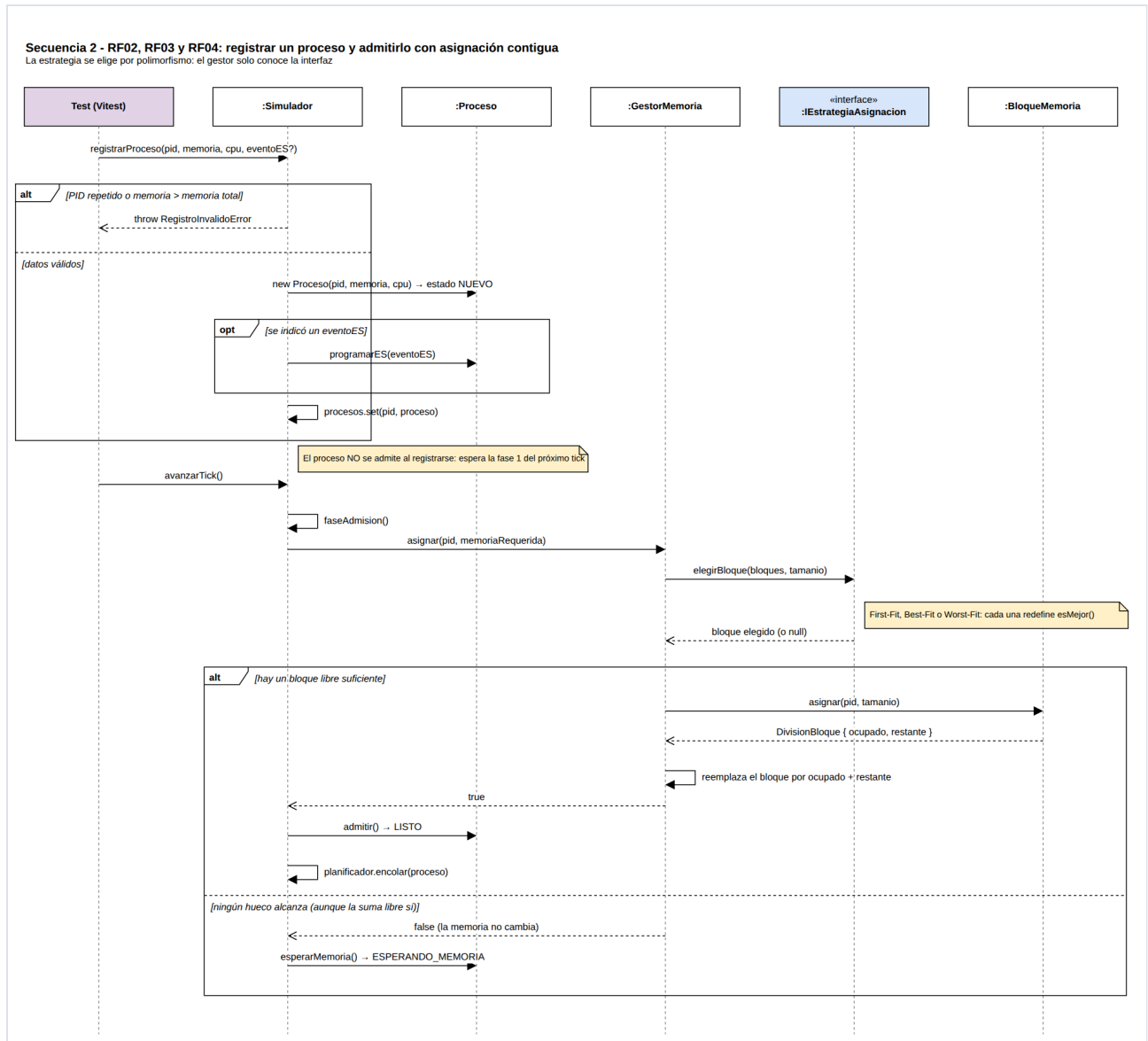


Figura 6. Secuencia 2 — RF02, RF03 y RF04: registro, admisión y asignación contigua (polimorfismo de la estrategia).

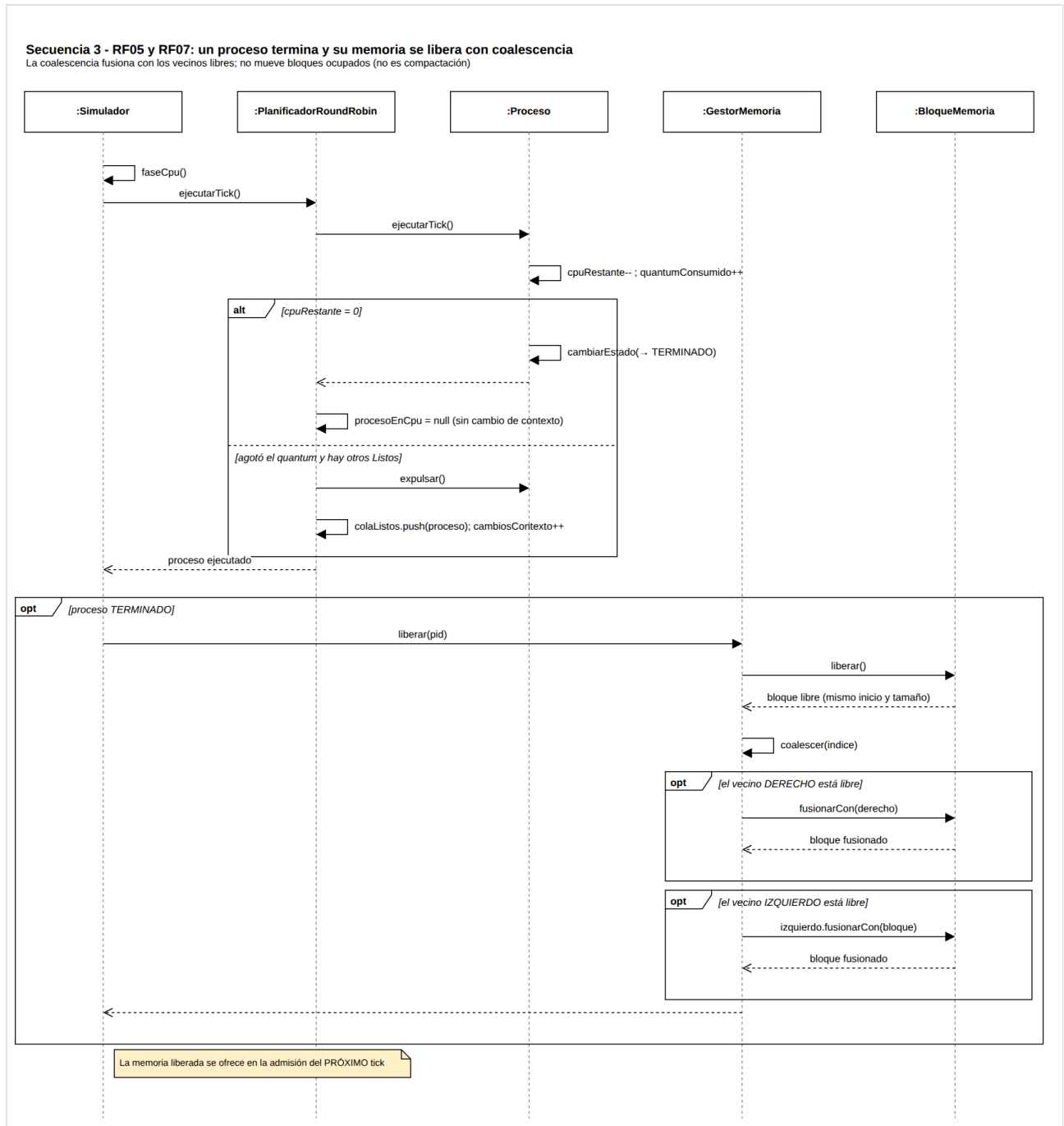


Figura 7. Secuencia 3 — RF05 y RF07: un proceso termina y su memoria se libera con coalescencia.

12. Matriz de trazabilidad RF → clase → tests

Requerimiento	Clase / método	Tests
RF01 Configurar e iniciar	Simulador (constructor), GestorMemoria, PlanificadorRoundRobin	Simulador.test.ts (configuración e inicio), Caso 1
RF02 Registrar y consultar	Simulador.registrarProceso, consultarProceso, listarProcesos; constructor de Proceso	Simulador.test.ts (registro, doble encapsulamiento), Proceso.test.ts (creación), Caso 1
RF03 Estados y admisión	Proceso (admitir, esperarMemoria, #cambiarEstado); Simulador.#faseAdmision	Proceso.test.ts (transiciones válidas e inválidas), Simulador.test.ts (fases), Casos 2 y 8
RF04 Memoria contigua	GestorMemoria.asignar, BloqueMemoria.asignar, estrategias	GestorMemoria.test.ts, BloqueMemoria.test.ts, EstrategiasAsignacion.test.ts, Caso 2
RF05 Liberar y coalescer	GestorMemoria.liberar, #coalescer, BloqueMemoria.fusionarCon	GestorMemoria.test.ts (coalescencia), BloqueMemoria.test.ts, Caso 3
RF06 Tick determinista	Simulador.avanzarTick y sus cuatro fases	Simulador.test.ts (fases de cada tick), Caso 8
RF07 Round Robin	PlanificadorRoundRobin.ejecutarTick, #resolverFinDeTick; Proceso.ejecutarTick, renovarQuantum, expulsar	PlanificadorRoundRobin.test.ts, Proceso.test.ts (ejecución de CPU), Casos 4 y 5
RF08 Entrada/Salida	EventoES, Proceso.programarES, avanzarBloqueo; Simulador.#faseBloqueados	EventoES.test.ts, Proceso.test.ts (E/S), PlanificadorRoundRobin.test.ts (bloqueo), Caso 6
RF09 Métricas	Metricas; Simulador.#faseRelojYMetricas	Metricas.test.ts, Caso 7
RF10 Estado del sistema	Consultas de Simulador (VistaProceso, copias de colas y mapa)	Simulador.test.ts (doble encapsulamiento), Caso 8 (invariantes)

Validación de eventos de E/S (RF08), documentada: la duración y el momento de disparo deben ser enteros positivos (DatoInvalidoError), y el evento se rechaza (EventoESInvalidoError) si nunca llegaría a dispararse (el proceso terminaría antes, porque la finalización tiene prioridad, o ese punto de CPU ya pasó) o si el proceso ya tiene un evento programado.

13. Casos de prueba y resultados

Hay **217 tests en 19 archivos**: tests unitarios por clase en `tests/src/` (normales, límites y errores) y **pruebas de colaboración** en `tests/colaboracion/CasosMinimos.test.ts`, que corren los ocho casos mínimos de la consigna sobre el simulador completo. Cada test tiene un comentario que indica qué caso o requerimiento verifica.

Caso mínimo	Datos de entrada	Resultado verificado
1. Configuración y registro	1024 KB, Q = 2; quantum 0, memoria -1; PID repetido; proceso de 2000 KB	Tick 0, un único bloque libre, CPU libre y colas vacías; todos los inválidos rechazados sin estado parcial
2. Asignación y espera	Procesos de 400 y 600 KB en 1000 KB; procesos de 600 KB que no entran juntos; huecos de 200, 500 y 174 KB con pedido de 150 KB	Partición parcial y ajuste exacto; fracaso sin modificar la memoria; admisión en el tick siguiente a la liberación; dirección 0 / 850 / 300 según la política
3. Coalescencia	4 procesos de 250 KB, Q = 1, con CPU que fuerza distintos órdenes de finalización	Fusión con el vecino izquierdo, con el derecho y con ambos; al final, un único bloque de 1000 KB
4. Round Robin	Q = 2, P1 con CPU 3, P2 con CPU 2	Traza P1, P1, P2, P2, P1 y 1 cambio de contexto
5. Quantum y finalización	Un proceso con CPU 5; dos procesos con CPU 2	Renueva el quantum sin cambio de contexto; terminar en el límite del quantum no reencola
6. Bloqueo por E/S	P1 (300 KB, CPU 4, E/S tras 1 tick, duración 2) y P2 (CPU 3)	Se bloquea conservando la memoria, no consume CPU, vuelve al final de Listos al vencer; 1 y luego 2 cambios de contexto
7. Métricas y límites	Huecos no contiguos de 100 y 300 KB; memoria llena; tick 0	Libre 400, mayor 300, fragmentación 25%; ocupación 100% y fragmentación 0%; utilización 0% en el tick 0
8. Orden e invariantes	Liberación y admisión en ticks consecutivos; escenario mixto de 30 ticks con E/S, espera de memoria y Best-Fit	La memoria liberada se usa recién en el tick siguiente; en todos los ticks: sin duplicados, sin solapamientos y como máximo un proceso en CPU

Captura: ejecución de las pruebas de colaboración

```

Fabricio@Fabricio-PC HING64 /d/UCP/AE2 - Paradigmas2 y SD/simulador-procesos-memoria (main)
$ npx vitest run tests/colaboracion --reporter=verbose

RUN v5.0.3 D:/UCP/AE2 - Paradigmas2 y SD/simulador-procesos-memoria

✓ tests/colaboracion/CasosMinimos.test.ts > Caso 1 - Configuración y registro > Estado inicial: tick 0, un único bloque libre, CPU libre, colas vacías y métricas en cero 3ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 1 - Configuración y registro > Rechaza parámetros inválidos, PID repetidos y procesos mayores que la memoria 1ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 2 - Asignación y espera > Partición parcial y ajuste exacto 1ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 2 - Asignación y espera > Fracaso sin modificación y admisión recién en el tick siguiente a la liberación 1ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 2 - Asignación y espera > Selección según política: con First-Fit el proceso de 150 KB va a la dirección 0 1ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 2 - Asignación y espera > Selección según política: con Best-Fit el proceso de 150 KB va a la dirección 850 0ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 2 - Asignación y espera > Selección según política: con Worst-Fit el proceso de 150 KB va a la dirección 300 0ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 3 - Coalescencia > Fusión con el vecino izquierdo, con el derecho y al final un único bloque 1ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 3 - Round Robin > Q=2, P1(CPU 3) y P2(CPU 2): se ejecuta P1, P1, P2, P2, P1 con 1 cambio de contexto 0ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 5 - Quantum y Finalización > Un único proceso renueva el quantum sin cambio de contexto 0ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 5 - Quantum y Finalización > Finalizar en el límite del quantum no reencola al proceso 0ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 6 - Bloqueo por E/S > Se bloquea, conserva la memoria, no consume CPU y vuelve a listos al vencer 1ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 7 - Métricas y límites > Huecos de 100 y 300 KB: libre 400, mayor 300 y fragmentación 25% 0ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 7 - Métricas y límites > Memoria llena: ocupación 100% y fragmentación 0% 0ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 7 - Métricas y límites > Tick 0: utilización de CPU 0% y sin fragmentación 0ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 8 - Orden e invariantes > La memoria liberada al final de un tick recién se usa en la admisión del siguiente 0ms
✓ tests/colaboracion/CasosMinimos.test.ts > Caso 8 - Orden e invariantes > En cada tick de un escenario mixto se cumplen todos los invariantes 8ms

Test Files 1 passed (1)
Tests 18 passed (18)
Start at 28:52:46
Duration 329ms (transform 68%, import 18%, tests 11%, worker 3%)

Fabricio@Fabricio-PC HING64 /d/UCP/AE2 - Paradigmas2 y SD/simulador-procesos-memoria (main)
$

```

Figura 8. Ejecución de `npx vitest run tests/colaboracion --reporter=verbose`: los 18 tests de los 8 casos mínimos aprobados.

14. Cobertura de código

Herramienta	Vitest 5 + @vitest/coverage-v8 (instrumentación nativa de V8)
Comando reproducible	npm ci y luego npm run coverage (equivalente a vitest run --coverage)
Alcance medido	Todos los archivos de src/**/*.ts, aunque ningún test los importe. Se excluye src/interfaces/ porque las interfaces solo declaran tipos y no generan código ejecutable.
Umbral	El comando falla si la cobertura de líneas es menor al 90% (thresholds.lines = 90 en vitest.config.ts).
Resultado (commit evaluado)	100% de líneas (304/304), ramas (130/130), funciones (115/115) y sentencias (312/312); 217 tests aprobados.
Integración continua	GitHub Actions corre typecheck y coverage en cada push y publica el reporte HTML como artefacto.

Captura: reporte en la terminal

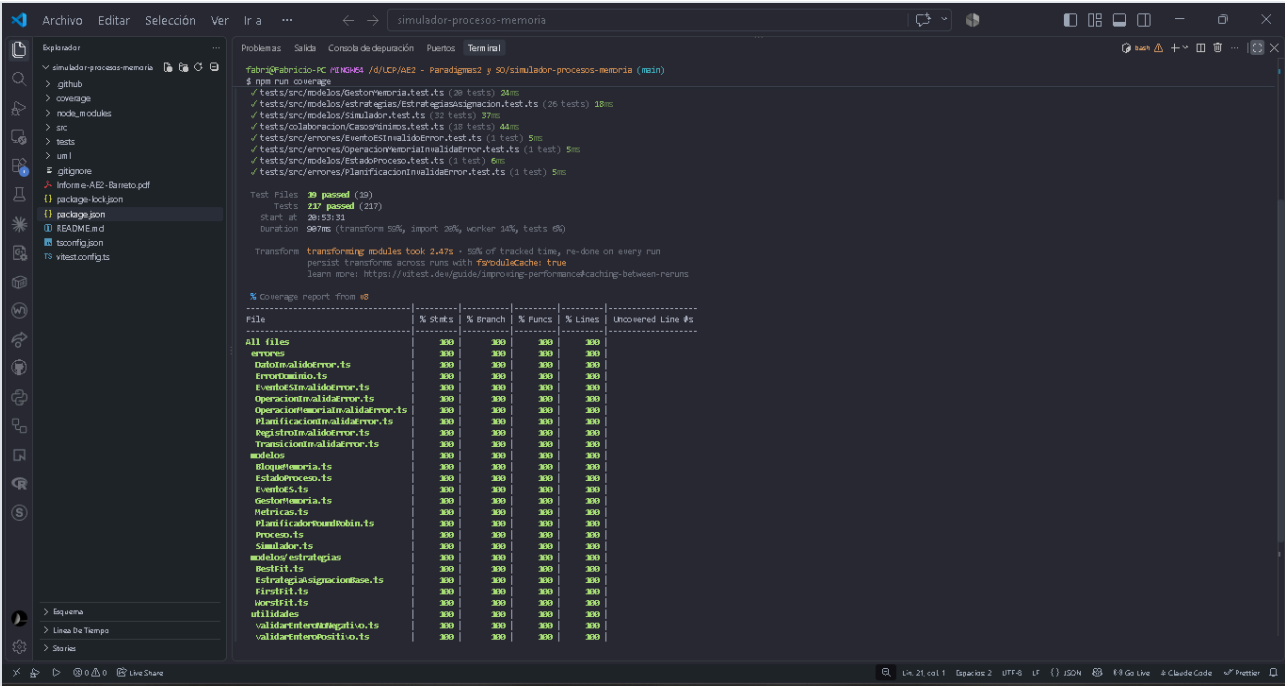


Figura 9. Salida de `npm run coverage`: 19 archivos, 217 tests aprobados y 100% de cobertura en todos los archivos.

Captura: reporte HTML (coverage/index.html)

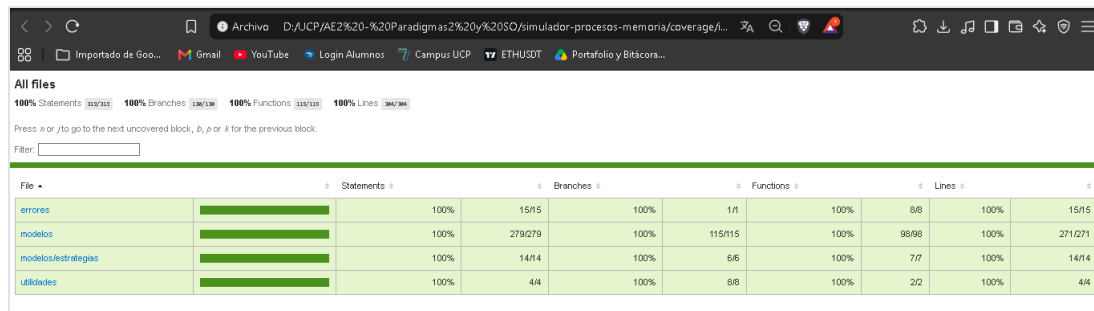


Figura 10. Reporte HTML de cobertura generado por `npm run coverage`.

15. Análisis comparativo First-Fit, Best-Fit y Worst-Fit

15.1 Metodología

Se corrieron los mismos escenarios con las tres políticas sobre la biblioteca, con 1024 KB de memoria y quantum 2, registrando cada proceso en su tick de llegada y avanzando hasta que todos terminaron. En cada tick se midió cuántos procesos esperaban memoria y la fragmentación externa (el promedio se toma sobre los ticks en que había memoria libre). Los escenarios aleatorios usan un generador con semilla fija, así que son deterministas y reproducibles.

- **A:** carga mixta de 12 procesos con llegadas escalonadas (uno o dos por tick).
- **C:** tres procesos largos intercalados con tres cortos dejan huecos de 200, 300 y 154 KB; en el tick 20 llega una ráfaga de seis procesos (150, 280, 190, 140, 250 y 100 KB).
- **D y E:** 40 procesos aleatorios (50–400 KB y 30–600 KB), uno de cada cuatro con mucha CPU, que llegan en ráfagas.

15.2 Resultados

Escenario	Política	Ticks hasta terminar	Ticks-proceso esperando memoria	Máx. en espera a la vez	Fragmentación externa promedio	Fragmentación máxima	Cambios de contexto
A - carga mixta con llegadas escalonadas	First-Fit	40	94	6	15.75%	47.17%	9
	Best-Fit	40	94	6	15.75%	47.17%	9
	Worst-Fit	40	94	6	15.75%	47.17%	9
C - huecos de 200, 300 y 154 KB y llega una ráfaga de 6 procesos	First-Fit	162	102	3	46.91%	56.47%	70
	Best-Fit	162	102	3	44.58%	56.47%	70
	Worst-Fit	162	108	2	44.50%	63.90%	70
D - carga aleatoria (40 procesos de 50 a 400 KB)	First-Fit	321	5255	31	29.56%	58.22%	125
	Best-Fit	321	5255	31	29.56%	58.22%	125
	Worst-Fit	321	5289	31	30.75%	58.22%	125
E - carga aleatoria (40 procesos de 30 a 600 KB)	First-Fit	201	2290	23	22.25%	68.25%	57
	Best-Fit	201	2290	23	22.25%	68.25%	57

Escenario	Política	Ticks hasta terminar	Ticks-proceso esperando memoria	Máx. en espera a la vez	Fragmentación externa promedio	Fragmentación máxima	Cambios de contexto
	Worst-Fit	201	2290	23	22.25%	68.25%	57

15.3 Foto del escenario C justo después de la ráfaga (tick 21)

Huecos disponibles antes de la ráfaga: 200 KB (dirección 150), 300 KB (dirección 450) y 154 KB (dirección 870).

First-Fit — esperan memoria: P9, P11, P12 · libre total 84 KB · mayor hueco 50 KB · fragmentación 40.48%

P1 150	P7 150	libre 50	P3 100	P8 280	libre 20	P5 120	P10 140	libre 14
-----------	-----------	-------------	-----------	-----------	-------------	-----------	------------	-------------

Best-Fit — esperan memoria: P10, P11, P12 · libre total 34 KB · mayor hueco 20 KB · fragmentación 41.18%

P1 150	P9 190	libre 10	P3 100	P8 280	libre 20	P5 120	P7 150	libre 4
-----------	-----------	-------------	-----------	-----------	-------------	-----------	-----------	------------

Worst-Fit — esperan memoria: P8, P11 · libre total 74 KB · mayor hueco 50 KB · fragmentación 32.43%

P1 150	P9 190	libre 10	P3 100	P7 150	P12 100	libre 50	P5 120	P10 140	libre 14
-----------	-----------	-------------	-----------	-----------	------------	-------------	-----------	------------	-------------

15.4 Interpretación

- **Las políticas solo se diferencian cuando hay varios huecos que alcanzan a la vez.** En el escenario A, y en E, los tres resultados son idénticos tick a tick: al haber un único hueco candidato (o al coincidir el primero, el más chico y el más grande), las tres eligen el mismo. La coalescencia inmediata y la admisión de a uno en orden de registro reducen mucho la cantidad de situaciones con varios candidatos.
- **Best-Fit** ocupa los huecos más ajustados (P7 de 150 KB en el hueco de 154 KB). Deja sobrantes diminutos (4, 10 y 20 KB) que casi no sirven para nada: en la foto, el mayor hueco libre es de solo 20 KB. Es la política que más memoria deja ocupada en ese instante, pero admite los mismos procesos que First-Fit.
- **Worst-Fit** usa el hueco más grande y deja sobrantes más aprovechables: en la ráfaga del escenario C **admitió cuatro procesos contra tres** de las otras dos políticas, y tuvo menos procesos esperando a la vez (máximo 2). Como contrapartida, rompe el hueco grande, por lo que un proceso grande que llega después (P8 de 280 KB) queda esperando; por eso su espera total fue un poco mayor (108 contra 102) y alcanzó la fragmentación máxima más alta (63,9%).
- **First-Fit** es el más simple y, en estas cargas, rindió igual o casi igual que Best-Fit. En la carga aleatoria D, First-Fit y Best-Fit coincidieron y Worst-Fit quedó apenas peor (más espera y más fragmentación promedio).
- En ningún escenario la política cambió el tiempo total ni los cambios de contexto: la CPU estuvo ocupada el 100% del tiempo y el cuello de botella fue la memoria.

Conclusión del análisis: con esta carga y este simulador, First-Fit es una buena opción por defecto: simple y con resultados equivalentes a Best-Fit. Best-Fit tiende a generar huecos inservibles y Worst-Fit favorece la admisión inmediata de procesos chicos a costa de los grandes.

16. Análisis de la fragmentación externa

La fragmentación externa se calcula como $100 \times (1 - \text{mayor hueco libre} / \text{memoria libre total})$, y vale 0% si no hay memoria libre. Mide qué parte de la memoria libre *no* se puede usar en un solo bloque contiguo.

- **Ejemplo de la consigna:** con huecos no contiguos de 100 y 300 KB hay 400 KB libres, pero el mayor hueco es de 300 KB: $100 \times (1 - 300/400) = 25\%$. Un proceso de 350 KB no entra aunque "sobren" 400 KB.
- **Cuándo aparece:** cuando terminan procesos que no son vecinos y quedan huecos separados por procesos que siguen ocupando memoria. En el escenario C, los procesos largos (P1, P3 y P5) actúan como "paredes" que impiden unir los huecos; la fragmentación promedio llegó a 44–47%.
- **Cómo influye la liberación:** cada liberación aplica coalescencia con los vecinos libres, lo que **reduce** la fragmentación (en el caso mínimo 3, liberar el proceso del medio une tres bloques en uno de 750 KB). Pero la coalescencia solo une huecos que ya están pegados: si entre dos huecos hay un proceso, siguen separados.
- **Reorganización:** la única forma de eliminar ese tipo de fragmentación sería la **compactación** (mover los procesos para juntar el espacio libre), que la consigna excluye expresamente porque tiene un costo alto de copia y de actualización de direcciones.
- **Efecto de la política:** Best-Fit deja huecos chicos dispersos; Worst-Fit consume los grandes y puede impedir que entren procesos grandes; First-Fit tiende a concentrar los huecos chicos al principio de la memoria.

17. Conclusiones

- Se construyó una biblioteca que cumple los diez requerimientos funcionales, verificada con 217 pruebas que pasan y 100% de cobertura del código de producción, sin interfaz ni salida por consola.
- Distribuir las responsabilidades (simulador que coordina, gestor de memoria, planificador, proceso, métricas) hizo que cada clase sea chica y que se pueda probar por separado; las pruebas de colaboración confirman que, juntas, respetan el orden de fases.
- El doble encapsulamiento (atributos privados reales, sin setters, operaciones que validan, copias y objetos inmutables) impide que desde afuera se rompan las reglas: la única forma de cambiar el estado es usar el simulador como corresponde.
- La combinación de interfaz, clase abstracta y subclasses para las políticas muestra el polimorfismo con un caso real: cambiar de First-Fit a Best-Fit es cambiar un objeto en la configuración.
- Desde Sistemas Operativos, la simulación permitió ver que la memoria contigua sufre fragmentación externa aunque se haga coalescencia, que la elección de política importa solo cuando hay varios huecos candidatos, y que el quantum y la E/S determinan los cambios de contexto.
- Trabajar con commits chicos, tests desde el principio e integración continua dio una línea de desarrollo verificable en cada paso.

18. Bibliografía y fuentes

- Silberschatz, A., Galvin, P. B. y Gagne, G. (2018). *Operating System Concepts* (10.^a ed.). Wiley. Capítulos de planificación de CPU y memoria principal.
- Tanenbaum, A. S. y Bos, H. (2015). *Modern Operating Systems* (4.^a ed.). Pearson. Capítulos de procesos y administración de memoria.
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9.^a ed.). Pearson.
- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley (patrones Strategy y Template Method).
- Martin, R. C. (2002). *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall (principios SOLID).
- Microsoft. *TypeScript Handbook*. <https://www.typescriptlang.org/docs/>
- Vitest. *Documentación oficial* (pruebas y cobertura). <https://vitest.dev/>
- GitHub. *GitHub Actions documentation*. <https://docs.github.com/actions>
- Consignas de la AE2 de Sistemas Operativos y de Paradigmas y Lenguajes de Programación II, UCP, 2026.

Anexo: cómo reproducir los resultados

```
git clone https://github.com/FabricioBarreto/simulador-procesos-memoria.git
cd simulador-procesos-memoria
git checkout v1.0
npm ci
npm run typecheck      # verificación de tipos
npm test               # 217 tests
npm run coverage       # tests + cobertura (reporte en coverage/index.html)
npx vitest run -t "Round Robin"  # un caso puntual
```

Modificaciones en vivo para la defensa: el quantum, la memoria y la política se cambian en la configuración del simulador dentro de un test (`new Simulador({ memoriaTotal: 2048, quantum: 3, estrategia: new WorstFit() })`); la liberación de un proceso determinado se provoca con `simulador.finalizarProceso(pid)`, que además dispara la coalescencia (test *"Finalizar procesos vecinos provoca la coalescencia en vivo"*).